

P/PC Audit

Production / Production Capability Health Check

Spring Web Service Sandbox

HEALTHY (for a sandbox)

Project: spring-web-service-sandbox

Stack: Java 25 · Spring Boot 4.0.3 · Hexagonal Architecture

Date: 9 March 2026

Period Assessed: Project lifetime (5 commits)

Step 1: Production (P) Summary

This project delivers a Book REST API sandbox built on Spring Boot 4.0.3 and Java 25 using hexagonal architecture. Based on git history (5 meaningful commits) and the codebase:

Features Shipped

- CRUD REST API for books (GET all, GET by ID, POST create) with JSON serialization
- Async bulk CSV import via RabbitMQ queue-based load leveling with lazy parsing and per-row validation
- MySQL persistence with Flyway schema migration
- Bean Validation on inbound requests (ISBN-13 regex, NotBlank constraints)
- Global exception handling for validation errors and IO failures
- OpenAPI/Swagger UI documentation (springdoc)
- Prometheus metrics with custom `@Timed` metric on import listener
- Pre-provisioned Grafana dashboard for import throughput monitoring
- Comprehensive actuator endpoint exposure
- Multi-layer test suite: unit (Mockito), slice (WebMvcTest), integration (Testcontainers)
- Docker Compose stack for all infrastructure dependencies
- 10MB multipart upload size limit

Not Delivered

UPDATE/DELETE endpoints, pagination, authentication/authorization, CI pipeline, production deployment artifacts (Dockerfile).

Step 2: Production Capability (PC) Assessment

Dimension	Status	Details
Tests	HEALTHY	Multi-layer coverage, good pyramid structure
Dependencies	WARNING	Bleeding-edge versions, compatibility concerns
Documentation	HEALTHY	Excellent README, clear architecture docs
Build/CI	WARNING	No CI pipeline whatsoever
Code Quality	HEALTHY	Clean hexagonal architecture, consistent patterns
Dev Experience	HEALTHY	Easy onboarding, clear conventions
Security	WARNING	Several issues beyond sandbox context
Observability	HEALTHY	Well-instrumented with Prometheus/Grafana

Tests — Healthy

4 test classes, 13 test methods across 3 layers:

- **Unit:** `BookServiceTest` (5 tests), `BookImportListenerTest` (1 test) — fast, no Spring context
- **Slice:** `BookControllerTest` (8 tests) with `@WebMvcTest` including validation edge cases
- **Integration:** `BookRepositoryTest` (2 tests) with Testcontainers MySQL; `SandboxApplicationTests` context load

Test Gaps

- No test for `BookRepositoryAdapter` mapping logic (`toBook` / `toEntity`)
- No end-to-end integration test for the full RabbitMQ import flow
- No test for `GlobalExceptionHandler` `IOException` path
- No negative test for malformed CSV files

Dependencies — Warning

- Spring Boot 4.0.3 and Java 25 are bleeding-edge — appropriate for sandbox but a production risk
- `springdoc-openapi-starter-webmvc-ui:3.0.2` needs Boot 4.x compatibility verification
- No security dependencies at all
- Lombok used minimally; domain model already uses records

Documentation — Healthy

Thorough README with architecture explanation, package-to-purpose tables, build instructions, Docker Compose usage, curl examples, Prometheus queries, endpoint tables. Better than most production projects.

Build/CI — Warning

- No CI pipeline of any kind (no `.github/workflows/`)
- Maven wrapper present — good for reproducible builds
- No build plugins beyond `spring-boot-maven-plugin` (no enforcer, spotbugs, jacoco)
- No Dockerfile for the application

Code Quality — Healthy

- Clean hexagonal architecture with proper dependency direction
- One minor impurity: `@Service` on `BookService` (Spring in domain layer)
- Records used correctly for value objects
- `open-in-view: false` — prevents lazy loading anti-pattern
- `ddl-auto: none` with Flyway — correct approach
- `JacksonJsonMessageConverter` restricts deserialization to trusted package

Security — Warning

- No authentication or authorization on any endpoint
- Actuator endpoints expose `env` , `beans` , `loggers` publicly
- `show-values: always` on local profile exposes all credentials
- Database credentials hardcoded in `application.yaml`
- No HTTPS, no rate limiting
- RabbitMQ uses default `guest/guest`

Observability — Healthy

- Prometheus metrics endpoint via actuator
- Custom `@Timed` annotation with `TimedAspect` bean
- Pre-built Grafana dashboard with 4 panels
- Prometheus scrape config at 15s intervals

- `InMemoryHttpExchangeRepository` for HTTP exchange tracking

Step 3: Diagnosis

HEALTHY (for a sandbox)

Sustainable pace. Production capability is proportional to production output.

The P/PC balance is appropriate for the stated purpose. Production output is focused and coherent. Production capability is adequate — tests cover the right layers, architecture is clean, documentation is thorough, Docker stack makes it easy to run.

The project is not “goose starving” because fundamental hygiene is in place. It is not “over-investing” because there is no gold-plating.

Note: If this project were intended for production use, the diagnosis would shift to **Warning** due to absent CI, security, and missing CRUD operations.

Step 4: Prescriptions

Priority	Finding	Action	Effort
1 (High)	No CI pipeline	Add GitHub Actions workflow running <code>./mvnw clean verify</code>	15 min
2 (Med)	No ISBN uniqueness constraint	Add <code>V2__add_isbn_unique.sql</code> migration + handle <code>DataIntegrityViolationException</code>	30 min
3 (Med)	Domain layer has Spring dependency	Move <code>@Service</code> to config class, or update README to acknowledge compromise	15 min
4 (Med)	No <code>BookRepositoryAdapter</code> test	Add unit test with mocked <code>JpaBookRepository</code> verifying mapping	20 min
5 (Low)	Actuator endpoints exposed	Add Spring Security or separate management port for sensitive endpoints	45 min

6 (Low)	No application Dockerfile	Add multi-stage Dockerfile + docker-compose service entry	30 min
7 (Low)	springdoc compatibility uncertain	Verify Swagger UI loads on Boot 4.x; upgrade if needed	10 min

Key Metrics Summary

Metric	Value
Source files	17 Java files (12 main + 5 test)
Test classes	4 (13 test methods)
Test layers	3 (unit, slice, integration)
Commits	5 meaningful commits
Dependencies	13 (including test scope)
Docker services	4 (MySQL, RabbitMQ, Prometheus, Grafana)
CI pipeline	None
Security dependencies	None
Architecture violations	1 (@Service in domain)